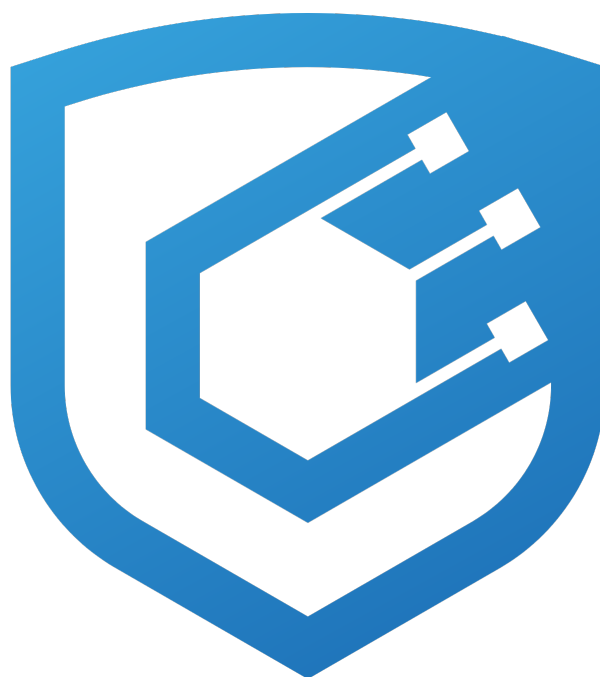




GreekFi Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[Alix40](#)

September 4, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
6.1	Centralization Risks	3
6.2	Operational risks	3
7	Findings	5
7.1	Medium Risk	5
7.1.1	Receipt::_payout lets attacker-chosen token granularity bypass all redemption fees	5
7.1.2	Post-deadline Receipt underpricing can cause premature liquidations	5
7.1.3	Receipt collateral has no liquidation exit before exerciseDeadline, letting a matched borrower push bad debt onto Morpho lenders	5
7.2	Low Risk	10
7.2.1	FactoryDeployer::deploy lets a front-runner invalidate a pre-mined vanity address	10
7.2.2	DeployMorphoVault sets the ETH feed max age equal to its heartbeat, so OracleReceipt::price reverts for a short window after most rounds	10
7.2.3	Receipt::setFee has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds	11
7.2.4	A single unrecoverable Receipt atom permanently disables sweep	12
7.2.5	Option::exercise uses the caller's live balance, unsolicited transfers can revert the call near expiry	12
7.3	Informational	14
7.3.1	DeployDeterministic::run does not verify the pinned Factory release	14
7.3.2	OracleReceipt::price wraps oversized call moneyiness into a negative signed value	14
7.3.3	OracleReceipt retains an unused commented-out SCALE declaration	14
7.3.4	Public mapping accessors omit key and value names	15
7.3.5	Unused declarations remain in Factory and Option	15
7.3.6	Receipt::redeemFor reverts an entire pre-deadline batch after the consideration pool is exhausted	15
7.3.7	IOption documents the wrong pair-burn event	16
7.3.8	Receipt::sweep can clear accrued fee accounting without reporting the cleared amount	16
7.3.9	FlashExerciseKeeper is illustrative but is not explicitly marked unsafe for production use	16
7.3.10	IRceipt::redeem amount overload NatSpec misstates the revert for amounts above the caller's balance	16
7.3.11	Index key fields in Exercise, Expire, and Swept events	17
7.3.12	The public Black-Scholes helper does not document its parameter units	17
7.3.13	OracleReceipt::price can publish zero at its moneyiness precision boundary	17
7.4	Gas Optimization	18
7.4.1	Receipt::collectFees loads the same fee accumulator twice	18
7.4.2	Factory::createOptions, createOptions2 copy read-only batch inputs into memory	18
7.4.3	Receipt::collectFees reads factory.owner() twice	18

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Greek.fi is a fully collateralized, oracle-free options protocol. Each market is an ERC20 pair of an `Option` (the long right to buy or sell collateral at a strike) and a `Receipt` (the short receipt for deposited collateral): users mint both sides 1:1 against collateral, an `Option` holder exercises by paying strike-denominated consideration to receive collateral, and a `Receipt` holder redeems consideration-first then collateral 1:1 on a purely time-gated schedule with no oracle ever consulted.

Each market is a non-upgradeable `Option` and `Receipt` pair created and registered by a `Factory` from fixed templates. The `Receipt` custodies the market's collateral and consideration, while the `Factory` is the shared allowance point: users approve it per token, and every registered `Receipt` can use that allowance for minting or exercising.

The owner cannot upgrade markets or directly sweep a funded `Receipt`, but can change its redemption fee without a cap or timelock, receive accrued fees, and sweep balances whenever its total supply is zero. Exercise and pair-burn remain fee-free.

5 Audit Scope

The audit scope was limited to:

```
contracts/Factory.sol
contracts/Option.sol
contracts/OracleReceipt.sol
contracts/Permissions.sol
contracts/Receipt.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [GreekFi](#) code provided by [GreekFi](#). The findings consist of 3 Medium & 5 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The factory owner sets the default fee applied to new markets (`Factory::setFee`, capped at 1000 bps) and is the sole payee of accrued protocol fees (`Receipt::collectFees`) and of residual balances swept only after a market is fully redeemed (`Receipt::sweep`).

The same owner can also reprice the redemption fee on any live market through `Receipt::setFee`, which is uncapped, so a high fee can skim or block that market's redemptions.

The owner cannot change a live market's collateral, strike, dates, tokens, or balances, and `Factory::renounceOwnership` is disabled, so the owner role can never be abandoned.

The `OracleReceipt` owner can replace the spot price source, volatility, and rate with no timelock (`OracleReceipt::setParams`), moving the mark used to price a `Receipt` posted as loan collateral in the Morpho integration.

No party can pause, freeze, seize, claw back, or upgrade positions: every option and receipt is a clone of an immutable template with no pause switch or emergency withdrawal, so centralization is confined to the fee powers above.

6.2 Operational risks

- `DeployMorphoVault` caps expiration at 45 days but does not cap the exercise window. Since oracle pricing and collateral redemption run until `exerciseDeadline`, deployments with longer windows must be reviewed against the full time horizon.

Summary

Project Name	GreekFi
Repository	contracts
Commit	65f9843ecd9a...
Fix Commit	23bfee78157f...
Audit Timeline	Aug 31st - Sep 2nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	5
Informational	13
Gas Optimizations	3
Total Issues	24

Summary of Findings

[M-1] Receipt::_payout lets attacker-chosen token granularity bypass all redemption fees	Resolved
[M-2] Post-deadline Receipt underpricing can cause premature liquidations	Resolved
[M-3] Receipt collateral has no liquidation exit before exerciseDeadline, letting a matched borrower push bad debt onto Morpho lenders	Partially Resolved
[L-1] FactoryDeployer::deploy lets a front-runner invalidate a pre-mined vanity address	Resolved
[L-2] DeployMorphoVault sets the ETH feed max age equal to its heartbeat, so OracleReceipt::price reverts for a short window after most rounds	Resolved
[L-3] Receipt::setFee has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds	Resolved
[L-4] A single unrecoverable Receipt atom permanently disables sweep	Resolved
[L-5] Option::exercise uses the caller's live balance, unsolicited transfers can revert the call near expiry	Resolved
[I-01] DeployDeterministic::run does not verify the pinned Factory release	Resolved
[I-02] OracleReceipt::price wraps oversized call moneyness into a negative signed value	Resolved
[I-03] OracleReceipt retains an unused commented-out SCALE declaration	Resolved
[I-04] Public mapping accessors omit key and value names	Resolved
[I-05] Unused declarations remain in Factory and Option	Resolved
[I-06] Receipt::redeemFor reverts an entire pre-deadline batch after the consideration pool is exhausted	Acknowledged
[I-07] IOption documents the wrong pair-burn event	Resolved
[I-08] Receipt::sweep can clear accrued fee accounting without reporting the cleared amount	Resolved
[I-09] FlashExerciseKeeper is illustrative but is not explicitly marked unsafe for production use	Resolved
[I-10] IReceipt::redeem amount overload NatSpec misstates the revert for amounts above the caller's balance	Resolved
[I-11] Index key fields in Exercise, Expire, and Swept events	Resolved
[I-12] The public Black-Scholes helper does not document its parameter units	Resolved
[I-13] OracleReceipt::price can publish zero at its moneyness precision boundary	Resolved
[G-1] Receipt::collectFees loads the same fee accumulator twice	Resolved
[G-2] Factory::createOptions, createOptions2 copy read-only batch inputs into memory	Resolved
[G-3] Receipt::collectFees reads factory.owner() twice	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `Receipt::_payout` lets attacker-chosen token granularity bypass all redemption fees

Description: `Receipt::_payout` calculates fees in the payout token's raw units and rounds down. Market creation is permissionless and imposes no minimum token decimals, so an attacker can deploy a zero-decimal wrapper whose single unit represents a valuable amount of an underlying asset.

For any fee below 100%, redeeming one unit produces a zero fee. Partial redemption lets the holder repeat one-unit redemptions, bypassing a fee that would be charged if the position were redeemed at once.

Ordinary rounding loses less than one raw unit per call. Here, the attacker controls the denomination and can make that raw unit economically valuable, turning marginal rounding into a complete fee bypass.

Impact: All redemption fees can be avoided in wrapper-denominated markets. The wrapper can be fully backed and redeemable, allowing genuine volume to be routed through the affected market. Fee loss is repeatable and scales with volume. Solvency and other holders' claims are unaffected.

Recommended Mitigation: Round nonzero fees up or carry fractional fees in accounting that cannot be reset through redemption splitting, token transfers, or new markets. Otherwise, restrict coarse token denominations or charge fees in a separate high-precision asset.

GreekFi: Fixed in [PR31](#)

Cyfrin: Verified. `Receipt` now rounds nonzero redemption fees up, so splitting redemptions or using a coarse token denomination cannot reduce the fee to zero.

7.1.2 Post-deadline `Receipt` underpricing can cause premature liquidations

Description: `OracleReceipt::price()` always values a `Receipt` as $1 - \text{optionPrice}$. After `exerciseDeadline`, it sets time to zero but continues valuing the expired exercise right at its current intrinsic value.

That is wrong for an unexercised `Receipt`. Assignment is closed, and when `consBacked == 0`, each `Receipt` is a fixed 1:1 collateral claim regardless of later spot movements.

A borrower can create this state by retaining the `Options` and supplying only the `Receipts` to Morpho. After the deadline, a spot movement can reduce the reported `Receipt` price even though the `Receipts` remain fully collateral-backed.

Morpho can then liquidate an adequately backed position using the incorrect price. An external liquidator can redeem the seized `Receipts` at par, transferring value from the borrower. Lenders are only exposed if the price gaps past the solvent liquidation range or liquidators fail to act.

Impact: The incorrect price can cause premature liquidation and loss for the borrower. With gradual price movement and active liquidators, the position becomes liquidatable while its collateral can still repay the debt in full. Lender loss requires a sufficiently large price gap or unavailable liquidators.

Recommended Mitigation: Return par after `exerciseDeadline` when `consBacked == 0`, without consulting the live spot price.

GreekFi: Fixed in [PR38](#).

Cyfrin: Verified. The oracle now returns par after the exercise deadline when `consBacked == 0`, so fully unexercised `Receipts` no longer follow the live spot price. Partially assigned series retain the documented conservative settlement mark as residual liquidation risk.

7.1.3 `Receipt` collateral has no liquidation exit before `exerciseDeadline`, letting a matched borrower push bad debt onto Morpho lenders

Description: Morpho lenders are protected only by liquidation. When a position falls below the threshold, a keeper repays the debt, seizes the collateral, and profits from the liquidation incentive ($\text{LIF} = \min(1.15, 1 / (1 - 0.3$

* (1 - LLTV))), 1.0438 at the deployment's 86% LLTV). The loan token is sUSDe and the seized collateral is Receipt tokens, so a keeper only liquidates if it can turn those Receipts into sUSDe in the same flow. There are only three ways a Receipt becomes an underlying token, and there is no DEX for them:

- `Option.burn` (pair-burn) pays sUSDe 1:1 and is open at any time, but it needs the matching `Option` tokens.
- `Receipt.redeem` before `exerciseDeadline` pays WETH, not sUSDe, and only up to `consBacked`, a single global first-come pool. The keeper then still has to sell that WETH for sUSDe, right after the move that triggered the liquidation.
- The collateral leg of `redeem` pays sUSDe 1:1, but only once `block.timestamp > exerciseDeadline`.

`Receipt::_redeem` shows the last two. The consideration leg is capped at `consBacked` (zero until someone exercises) and the collateral leg is gated on the deadline, so while a series is unassigned every `redeem` reverts `ExerciseWindowOpen`:

```
function _redeem(address account, uint256 amount_) internal nonZero(amount_) {
    if (isEuro() && block.timestamp < expirationDate()) revert BeforeExerciseWindow();
    uint256 amount = amount_ > consBacked ? consBacked : amount_;
    uint256 remaining = block.timestamp > exerciseDeadline() ? amount_ - amount : 0;
    uint256 consAmount = toConsideration(amount, false);
    if (amount == 0 && remaining == 0) revert ExerciseWindowOpen();
    consBacked -= amount;

    if (amount > 0 && consideration().balanceOf(address(this)) < consAmount) revert InsufficientPool();
    if (remaining > 0 && collateral().balanceOf(address(this)) < remaining) revert InsufficientPool();

    uint256 rtBurned = amount + remaining;
    _burn(account, rtBurned);

    uint256 considerationPaid = _payout(consideration(), account, consAmount);
    uint256 collateralPaid = _payout(collateral(), account, remaining);
    emit Redeemed(msg.sender, account, rtBurned, collateralPaid, considerationPaid);
}
```

So calling `liquidate` is not enough. A keeper needs path 1 or 3, or a raced slice of 2. A stranger who seizes the Receipts has none of these before the deadline. It holds no `Option`, so path 1 is closed. Path 3 is gated until the deadline. Path 2 pays the wrong asset, is capped at `consBacked`, and reverts while `consBacked` is zero. Off-chain there is no dependable venue either. Receipts trade only through discretionary request-for-quote market making, and a maker who buys a seized Receipt inherits the same lockup, so it will not quote near the mark for a bespoke series or during the move that triggered the liquidation. (It is also important to note, liquidation bot usually require onchain solutions to execute liquidations and not off-chain)

The minter who kept the `Option` tokens is the exception. `Option.burn` pair-burns a Receipt with its `Option` for sUSDe 1:1 at any time:

```
// Option
function burn(address account, uint256 amount) public nonReentrant nonZero(amount) {
    if (notAuthorized(account, msg.sender, Perm.BURN)) revert Unauthorized();
    _burn(account, amount);
    receipt.burn(account, amount);
    emit PairBurned(msg.sender, account, amount);
}

// Receipt
function burn(address account, uint256 amount) public onlyOption nonReentrant {
    _burn(account, amount);
    collateral().safeTransfer(account, amount);
}
```

So the minter self-liquidates. It supplies the Receipts to Morpho, keeps the Options, borrows sUSDe, and waits. Once a normal price move drops the mark below $LLTV * LIF$ (0.898 of the mark at borrow time), it calls `liquidate` on its own position, receives the Receipts in the callback, pair-burns them with the retained Options for par sUSDe,

repays only $\text{collateral} * \text{mark} / \text{LIF}$, and Morpho writes off the rest. An isolated mint of any size is the whole proof, and a roughly 20% adverse move by mid-life crosses the threshold.

The bug does not depend on the mark being wrong. Whether or not the mark is exactly right, no stranger can convert a live, unassigned Receipt into the sUSDe borrow token, while the matched minter can. The loss comes from the collateral being unrealisable by anyone else, not from a pricing error.

This survives the objection that an ETH dump makes the puts exercise, which raises `consBacked` and opens path 2 for keepers. That does not save lenders when the borrower is also the dominant writer. Suppose one wallet holds most of the Receipts, supplied to Morpho, and the matching Options, and does not sell the longs. Anyone can call `liquidate` once the mark is below the threshold, but nobody can realise that wallet's Receipts as sUSDe. Outsiders hold only the minority of Options, so they can push `consBacked` to at most their own share and cannot assign the wallet's position. A stranger who seizes the wallet's Receipts can redeem at most that minority as WETH, and only if those outsiders actually exercise and the free Receipt holders do not drain the pool first. If the free holders take the WETH pool, `consBacked` returns to zero and the seized Receipts are fully stuck. The wallet is the only party that can burn them for sUSDe, so it self-liquidates and the write-off is lender loss.

The loss scales with the market's supply, not the 1,000 sUSDe cap, which only limits the MetaMorpho vault. Direct Morpho suppliers are uncapped, and the borrower can loop mint, supply, and borrow for leverage of about 4.6x at the ATM 30-day mark used above, up to roughly 7x when the mark is near par. Among MetaMorpho v1.1 vault depositors the loss is not even shared, because the vault does not mark shares down on a market loss, so shares redeem at par until the vault runs dry and the last depositor to redeem absorbs the shortfall. With two 500-sUSDe depositors and the 18.72 sUSDe write-off from the 20% path above, the first to redeem gets out whole and the last one absorbs the 18.72. Direct Morpho Blue suppliers, by contrast, socialise the loss pro-rata.

Impact: Morpho lenders lose loan principal as bad debt, because no third party can reliably realise the seized Receipt collateral while the option is live, whereas the matched borrower can always recover par.

Proof of Concept: Add the following file to `test/` and run `forge test --match-contract PreDeadlineNoExit -vv`. The first test shows a liquidator stand-in cannot exit while the matched holder pair-burns for par. The other two print the mark path and assert the bad-debt threshold is crossed.

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.30;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { OracleReceiptTest } from "../OracleReceipt.t.sol";
import { Option } from "../contracts/Option.sol";
import { Receipt as Rct } from "../contracts/Receipt.sol";
import { IReceipt } from "../contracts/interfaces/IReceipt.sol";

contract PreDeadlineNoExit is OracleReceiptTest {
    uint256 constant LLTV_WAD = 0.86e18;

    function _lif() internal pure returns (uint256) {
        // Morpho Blue: min(1.15, 1 / (1 - 0.3 * (1 - LLTV)))
        uint256 denom = 1e18 - (0.3e18 * (1e18 - LLTV_WAD)) / 1e18;
        uint256 lif = 1e36 / denom;
        return lif > 1.15e18 ? 1.15e18 : lif;
    }

    function _threshold(uint256 mark0) internal pure returns (uint256) {
        return mark0 * LLTV_WAD / 1e18 * _lif() / 1e18;
    }

    function test_PreDeadline_ThirdPartyCannotRealise_MatchedHolderCan() public {
        uint256 amount = 1_000e6;
        IReceipt receipt = putOracle.r();
        Option putOption = Option(address(receipt.option()));
        Rct rct = Rct(address(receipt));

        // attacker = this: mints the matched pair, keeps the Options, hands the receipts to a
        // liquidator stand-in
    }
}
```

```

usdc.mint(address(this), amount);
usdc.approve(address(factory), amount);
putOption.mint(amount);

address liquidator = makeAddr("liquidator");
IERC20(address(rct)).transfer(liquidator, amount);

// 15 days in, still pre-expiry (expiry = +30d): the liquidator holding receipts cannot exit
vm.warp(block.timestamp + 15 days);
assertEq(receipt.consBacked(), 0);
vm.prank(liquidator);
vm.expectRevert(IReceipt.ExerciseWindowOpen.selector);
rct.redeem();
vm.prank(liquidator);
vm.expectRevert(IReceipt.ExerciseWindowOpen.selector);
rct.redeem(1);

// matched holder: receipts back with the attacker, pair-burn for par at any time
vm.prank(liquidator);
IERC20(address(rct)).transfer(address(this), amount);
uint256 before = usdc.balanceOf(address(this));
putOption.burn(amount);
assertEq(usdc.balanceOf(address(this)) - before, amount, "pair-burn returns par pre-deadline");
}

/// Entry A: borrow at the ATM mark 30 days out, how far must spot fall by mid-life
function test_PreDeadline_EntryAtm30d() public {
    uint256 mark0 = putOracle.price();
    uint256 thr = _threshold(mark0);
    emit log_named_decimal_uint("A mark0 (ATM, 30d)", mark0, 36);
    emit log_named_decimal_uint("A bad-debt threshold", thr, 36);
    vm.warp(block.timestamp + 15 days);
    uint256[3] memory spots = [uint256(1700e18), 1600e18, 1500e18];
    for (uint256 i; i < 3; ++i) {
        source.setSpot(spots[i]);
        uint256 m = putOracle.price();
        emit log_named_decimal_uint(i == 0 ? "A mark, spot -15%, 15d left " : i == 1 ? "A mark,
            ↪ spot -20%, 15d left " : "A mark, spot -25%, 15d left ", m, 36);
    }
    source.setSpot(1600e18);
    assertLt(putOracle.price(), thr, "20% drop mid-life creates bad debt on a max-LTV position
        ↪ opened ATM");
}

/// Entry B: borrow 2 days before expiry with the put 5% OTM (mark near par), spot then slides into
    ↪ the deadline
function test_PreDeadline_EntryOtmNearExpiry() public {
    vm.warp(expiry - 2 days);
    source.setSpot(2100e18);
    uint256 mark0 = putOracle.price();
    uint256 thr = _threshold(mark0);
    emit log_named_decimal_uint("B mark0 (5% OTM, 2d)", mark0, 36);
    emit log_named_decimal_uint("B bad-debt threshold", thr, 36);
    vm.warp(deadline - 1);
    uint256[3] memory spots = [uint256(1900e18), 1800e18, 1700e18];
    for (uint256 i; i < 3; ++i) {
        source.setSpot(spots[i]);
        uint256 m = putOracle.price();
        emit log_named_decimal_uint(i == 0 ? "B mark, spot 0.95K, deadline " : i == 1 ? "B mark,
            ↪ spot 0.90K, deadline " : "B mark, spot 0.85K, deadline ", m, 36);
    }
    source.setSpot(1750e18);

```

```

    assertLt(putOracle.price(), thr, "a ~17% slide from 5% OTM to 12.5% ITM into the deadline
    ↳ creates bad debt");
  }
}

```

The full end-to-end attack was reproduced against live Morpho on a mainnet fork with the option still live. Borrow 780 sUSDe against a 1,000-collateral position at the ATM 30-day mark (0.9105), warp 15 days so roughly 15 days remain before the deadline, move spot 20% adverse so the mark falls to 0.7957 below the 0.8173 threshold, then self-liquidate. The borrower seizes all 1,000 collateral, pair-burns it with the retained Options for full par in `onMorphoLiquidate`, repays only 762.29, and Morpho writes off 18.72 sUSDe of bad debt against the suppliers, all before `exerciseDeadline`. A third party holding the seized Receipt at that instant reverts `ExerciseWindowOpen` on `redeem`.

Recommended Mitigation: There is no reliable on-chain way for a third party to realise a seized Receipt in the loan token before `exerciseDeadline`, so the practical mitigation is operational. None of the following requires a contract change.

- Keep the markets experimental and curated. Do not open them to public or permissionless lenders.
- Create each lending series with a short maturity. A keeper that warehouses waits from the liquidation to `exerciseDeadline`, so a shorter maturity means less time exposed with no third-party exit. Keep `windowSeconds` small as well, so the deadline stays close to expiry and a series cannot be created with one far past expiry that stretches the wait and inflates the oracle's time-to-expiry. The deployed American puts already use a short window.
- Create the Morpho market with a conservative LLTV, sized to the adverse move you are willing to absorb over the series, since a lower LLTV pushes the bad-debt line $\text{mark0} * \text{LLTV} * \text{LIF}$ further down. Morpho allows only enabled LLTV values and the market is immutable, so this is a creation-time choice. On the PoC's 20% path the enabled 62.5% avoids bad debt where 86% does not, and a deeper crash still breaches it.
- Keep the redemption fee at or below the documented 1% and never raise it on a live market. A fee above the near-par break-even, about 420 bps at 86% LLTV ($1 - 1 / \text{LIF}$), makes a third-party redeem-at-the-mark unprofitable. Pair-burn pays no fee, so a high fee never slows the self-liquidator.
- Rely on a warehousing keeper. At a low LLTV this is profitable enough that a specialist willing to lock capital to the deadline will do it, though ordinary Morpho flash liquidators will not, since they need sUSDe in the same transaction. It liquidates at the first crossing, holds the seized Receipt to `exerciseDeadline`, and redeems. The liquidation bonus, about 12.7% at 62.5% LLTV against 4.4% at 86%, is the return if the keeper realises at the mark, and it covers the roughly 1.2% cost of capital over a 45-day wait with wide margin. In the common unassigned case the keeper does better: it pays about mark / LIF (0.81 at mark 0.91 and 62.5%) and redeems the collateral leg at par minus the fee (0.99 at a 1% fee), about a 22% total on the hold. A first-crossing liquidation repays lenders in full, and losing that race to the borrower still leaves lenders whole, since the borrower only takes the bonus. So run or fund a specialist keeper for these markets.
- Monitor position health against the mark, feed freshness, and `consBacked`, so the keeper can act at the first crossing.

Residuals to document for anyone supplying to these markets. The exit still depends on a keeper being present and winning the race to the first crossing, which is not guaranteed for a thin market. A gap through both the liquidation and bad-debt lines in one oracle update, or a stale feed that reverts `price` and freezes liquidation, can still leave bad debt. Keepers need standing capital that a large simultaneous move can exhaust. The fee is an owner setting, so keeping it low relies on operator discipline. Direct Morpho supply is not capped by the vault.

Before any public or permissionless use these markets need a reliable third-party exit, which operations cannot supply and which is a larger design change outside this guidance. Do not rely on a discretionary off-chain options market, and do not use an escrowed-`Option` collateral tier. **Greekfi:** Risk is documented and warned against

Cyfrin:

7.2 Low Risk

7.2.1 FactoryDeployer::deploy lets a front-runner invalidate a pre-mined vanity address

Description: FactoryDeployer::deploy is permissionless, and owner is not included in the CREATE2 commitment. A searcher can copy a pending deployment's initCode and salt, deploy first with a different owner, and occupy the expected vanity address.

The official deployment script detects the collision and reverts, so the hostile deployment is not accepted as official.

Impact: An attacker can delay a new-chain rollout and force the team to mine a new vanity address. They cannot take control of a Factory accepted by the official deployment process. The impact is limited to deployment griefing.

Recommended Mitigation: Submit deployments privately and withhold salts until confirmation. Future versions should bind the owner into the CREATE2 commitment or require signed deployment authorization.

GreekFi: Fixed in [PR32](#)

Cyfrin: Verified. FactoryDeployer now binds the intended owner into the CREATE2 salt, so a caller using a different owner derives a different address.

7.2.2 DeployMorphoVault sets the ETH feed max age equal to its heartbeat, so OracleReceipt::price reverts for a short window after most rounds

Description: DeployMorphoVault wires ChainlinkCrossPriceSource with UNDERLYING_MAX_AGE = 3600 for the ETH/USD feed, and the comment describes the value as "heartbeat + grace":

```
address constant ETH_USD_FEED = 0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419;
address constant SUSDE_USD_FEED = 0xFF3BC18cCbD5999CE63E788A1c250a88626aD099;
uint256 constant UNDERLYING_MAX_AGE = 3600; // heartbeat + grace
uint256 constant CASH_MAX_AGE = 90000; // 25h
```

```
ChainlinkCrossPriceSource source = new ChainlinkCrossPriceSource(
    AggregatorV3Interface(ETH_USD_FEED), AggregatorV3Interface(SUSDE_USD_FEED), UNDERLYING_MAX_AGE,
    ↪ CASH_MAX_AGE
);
```

3600 s is exactly the ETH/USD heartbeat, with no grace. Chainlink's heartbeat round lands a few seconds late in calm markets, and ChainlinkCrossPriceSource::_read rejects any answer older than maxAge with a strict comparison:

```
function _read(AggregatorV3Interface feed, uint256 maxAge) internal view returns (uint256) {
    (, int256 answer, , uint256 updatedAt,) = feed.latestRoundData();
    if (answer <= 0) revert InvalidPrice(address(feed));
    if (updatedAt == 0 || updatedAt > block.timestamp || block.timestamp - updatedAt > maxAge) {
        revert StalePrice(address(feed));
    }
    return uint256(answer);
}
```

Sampled on mainnet on 2026-09-01 from the live aggregator behind the ETH/USD proxy (rounds 33173 to 33193): 11 of the last 20 inter-round gaps were 3612 to 3636 s. During the 12 to 36 s after each such round `block.timestamp - updatedAt > 3600` holds and every `price()` call reverts `StalePrice`. The sUSDe/USD leg is configured correctly (24 h heartbeat, 25 h max age).

Impact: Morpho borrow, liquidate, and debt-side withdrawCollateral revert for a 12-36 second window after most ETH/USD heartbeat rounds.

Recommended Mitigation: Set the underlying max age to the heartbeat plus a real grace period and make the comment true, for example:

```
uint256 constant UNDERLYING_MAX_AGE = 3600 + 600; // heartbeat + grace
```

Chainlink's guidance is to allow a margin above the heartbeat. A value of 3900 to 4200 s keeps the staleness guard meaningful without tripping on ordinary round latency.

GreekFi: Fixed in [PR40](#)

Cyfrin: Verified. The deployment script now allows a ten-minute grace period above the ETH/USD feed heartbeat.

7.2.3 Receipt::setFee has no upper bound, letting the Factory owner take 100% of a live market's redemption proceeds

Description: Receipt::setFee is callable by the Factory owner on any live market and stores feeBps_ after a caller check only:

```
/// @inheritdoc IReceipt
function setFee(uint64 feeBps_) external {
    if (msg.sender != address(factory) && msg.sender != factory.owner()) revert UnauthorizedCaller();
    feeBps = feeBps_;
    emit FeeUpdated(feeBps_);
}
```

Factory::setFee caps the creation-time default at 1000 bps, and Factory::createOption2 seeds each new market from that capped value via Receipt(receipt_).setFee(feeBps), but the per-market setter never re-applies the bound:

```
/// @inheritdoc IFactory
function setFee(uint64 bps) external onlyOwner {
    if (bps > 1000) revert InvalidValue();
    feeBps = bps;
    emit Fee(bps);
}
```

Receipt::_payout computes the fee and the net payout on both legs of every redemption:

```
function _payout(IERC20 token, address account, uint256 amount) internal returns (uint256 paid) {
    if (amount == 0) return 0;
    uint256 fee = Math.mulDiv(amount, feeBps, 10_000);
    if (fee > 0) feeAccrued[address(token)] += fee;
    paid = amount - fee;
    token.safeTransfer(account, paid);
}
```

At feeBps == 10_000 the redemption succeeds after Receipt::_redeem has burned the holder's receipts, pays 0, and credits the full gross amount to feeAccrued, which Receipt::collectFees pays to factory.owner(). Above 10_000 the subtraction underflows and every Receipt::redeem, redeemFor call on that market reverts until the owner lowers the fee. Pair-burn still works, but a writer who has sold their longs has no exit.

The IReceipt::setFee NatSpec acknowledges the missing cap:

```
/// @notice Set this market's redeem fee. Callable by the creating Factory during deployment or
///         by the current Factory owner afterward. No cap or timelock is applied; values above
///         10000 bps make positive redemptions revert.
function setFee(uint64 feeBps_) external;
```

but the documented trust model says the opposite. The IFactory::owner NatSpec limits the owner's setFee reach to 10% and states it cannot touch a funded pool, and the README's "Trust boundaries" section and the Factory constructor note repeat the same guarantee:

```
/// @notice The {Ownable} owner. Its reach into the protocol: {setFee} ( 10 %, used as the
///         default during market initialization; {IReceipt.setFee} can reprice a live market),
///         {IReceipt.sweep} (gated on `totalSupply() == 0`) and receiving {IReceipt.collectFees}.
///         It cannot touch a live position or a funded pool. `renounceOwnership` is disabled
///         ({OwnershipNotRenounceable}) because an ownerless factory would strand every accrued
```

```

/// fee and sweepable balance in every Receipt it ever created.
/// @return The current owner; never `address(0)`.
function owner() external view returns (address);

```

The code implements the weaker claim.

Impact: The Factory owner can take 100% of a live, funded market's redemption proceeds, or halt its redemptions entirely, with a single fee update - contradicting the documented "cannot touch a funded pool" and " $\leq 10\%$ " owner-reach limits.

Recommended Mitigation: Enforce the same bound as `Factory::setFee` so the per-market rate can never exceed the documented 10%:

```

error InvalidFee(); // add to IReceipt

function setFee(uint64 feeBps_) external {
    if (msg.sender != address(factory) && msg.sender != factory.owner()) revert UnauthorizedCaller();
    if (feeBps_ > 1000) revert InvalidFee();
    feeBps = feeBps_;
    emit FeeUpdated(feeBps_);
}

```

GreekFi: Fixed in [PR39](#)

Cyfrin: Verified. Receipt now enforces the same 1,000 bps fee ceiling as Factory.

7.2.4 A single unrecoverable Receipt atom permanently disables sweep

Description: `Receipt::sweep` is the only way to recover tokens held above the amount required to back Receipt holders, but it reverts whenever `totalSupply() != 0`.

Receipts can be transferred to any address, including the Receipt contract itself. If a holder sends one Receipt atom there, the contract cannot redeem or burn it, so total supply can never return to zero. Every later sweep then reverts, even for balances that are provably unrelated to holder claims, such as donations, rounding residue, or accepted transfer over-delivery.

Impact: A griever can permanently disable surplus recovery for the cost of one Receipt atom and gas. The attacker cannot withdraw the stranded assets, and normal holder backing and redemptions remain solvent, so the impact is limited to a cheap denial of the rescue mechanism.

Recommended Mitigation: Allow the owner to sweep only the portion of the collateral or consideration balance that exceeds the amount required to back outstanding Receipts. Keep other tokens gated on zero supply and document that tokens sharing a balance ledger are unsupported.

GreekFi: Fixed in [PR47](#)

Cyfrin: Verified. Receipt now limits live-supply sweeps to balances above the collateral and consideration backing required by outstanding Receipts, so a stranded Receipt atom no longer disables recovery of fees, donations, or foreign tokens.

7.2.5 `Option::exercise` uses the caller's live balance, unsolicited transfers can revert the call near expiry

Description: The no-argument `Option::exercise` exercises the caller's full live balance. Anyone can transfer Options to the holder before the transaction. If the holder approved only enough consideration for their original balance, the added Options increase the pull in `Receipt::exercise`, causing the call to revert.

Impact: An attacker can delay exercise for the cost of the transferred Options. The holder can retry with `exercise(uint256)`, but a last-minute revert may make them miss the exercise window and lose the option's in-the-money value.

Recommended Mitigation: Document that the no-argument overload uses an externally inflatable balance and recommend `exercise(uint256)` for deadline-sensitive calls. Alternatively, make it best-effort by capping the exercise to the consideration the holder can currently spend:

```
uint256 optionBalance = balanceOf(msg.sender);
IERC20 cons = receipt.consideration();
uint256 budget = Math.min(
    cons.allowance(msg.sender, address(FACTORY)),
    cons.balanceOf(msg.sender)
);
uint256 amount = receipt.toConsideration(optionBalance, true) <= budget
    ? optionBalance
    : receipt.toCollateral(budget);
exerciseFor(msg.sender, amount);
```

Return the exercised amount and document that this overload may partially exercise.

GreekFi: Fixed in [PR46](#)

Cyfrin: Verified. The no-argument exercise documentation now warns that unsolicited transfers can increase the live balance and recommends the fixed-amount overload for deadline-sensitive calls.

7.3 Informational

7.3.1 `DeployDeterministic::run` does not verify the pinned `Factory` release

Description: `DeployDeterministic::run` hashes the locally compiled `Factory` creation code, predicts an address from that hash, deploys the same creation code, and then requires the returned address to equal the prediction. This equality confirms only that `CREATE2` behaved as expected. It holds for any locally compiled creation code and does not establish that the code matches the init-code hash or address pinned for the intended release.

Impact: An operator using the wrong source revision, compiler configuration, or linked library address can successfully deploy a `Factory` at an unintended address while every in-script assertion passes. Integrations configured for the release's canonical address will not use that deployment, requiring a corrected deployment and potentially causing operational confusion or cross-chain address drift.

Recommended Mitigation: Before broadcasting, require the locally computed init-code hash to equal a release-pinned hash and require the prediction to equal an explicitly supplied or release-pinned expected `Factory` address. Keep these expected values in a versioned deployment artifact rather than deriving both sides of the checks from the current local build.

GreekFi: Fixed in [PR32](#)

Cyfrin: Verified. The deployment script now checks the locally compiled `Factory` hash and predicted address against independently pinned release values before broadcasting.

7.3.2 `OracleReceipt::price` wraps oversized call moneyness into a negative signed value

Description: `Factory::createOption2` permits an arbitrarily large nonzero strike when the consideration token has no more decimals than the collateral token. If an integration attaches `OracleReceipt` to such a call market, `OracleReceipt::price` can compute moneyness above `type(int256).max`. The pure pricing boundary then converts that unsigned value to `int256`, wrapping it to a negative number before the logarithm and intrinsic-value calculations.

Impact: Pricing a deliberately extreme call market can revert while the market is live and can return zero or revert after the exercise deadline, depending on the wrapped value. This fails closed rather than overvaluing collateral, and the documented Morpho deployment accepts only puts, so the supported production profile is not exposed. Generic integrations that use `OracleReceipt` for call markets can nevertheless receive a broken mark.

Recommended Mitigation: Reject moneyness above `type(int256).max` before the cast in `OracleReceipt::price`. An integration that supports only a narrower market profile should also validate the Receipt's flavor and strike range before deploying the oracle.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. `OracleReceipt` now rejects moneyness values above the signed-integer range before casting them.

7.3.3 `OracleReceipt` retains an unused commented-out `SCALE` declaration

Description: `OracleReceipt` contains a commented-out `SCALE` constant declaration that is not part of the compiled pricing implementation. Retaining dead code beside active scale documentation can make it less clear which scaling factor the implementation actually uses.

Recommended Mitigation: Delete the commented-out declaration while retaining the explanatory comment about why the Morpho scale collapses to `1e18`.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. The unused commented `SCALE` declaration has been removed from `OracleReceipt`.

7.3.4 Public mapping accessors omit key and value names

Description: The public `Factory::receipts`, `optionFor`, `permissions` and `Receipt::feeAccrued` mappings omit named key and value parameters. Their generated getters therefore expose less descriptive ABI metadata than the surrounding public interface.

Recommended Mitigation: Add descriptive key and value names to each mapping declaration, including both key levels of permissions.

GreekFi: Fixed in [PR35](#)

Cyfrin: Verified. The public mapping declarations now name their keys and return values, including both levels of permissions.

7.3.5 Unused declarations remain in `Factory` and `Option`

Description: `Factory::createOption` declares the named return variable `option_` but returns the result of `createOption2` directly without assigning that variable. Separately, `Option` imports `IERC20` without using the symbol. These declarations add noise without affecting the compiled behavior.

Recommended Mitigation: Make the `Factory::createOption` return value unnamed and remove the unused `IERC20` import from `Option`.

GreekFi: Fixed in [PR33](#)

Cyfrin: Verified. The unused `createOption` return name and unused `IERC20` import have been removed.

7.3.6 `Receipt::redeemFor` reverts an entire pre-deadline batch after the consideration pool is exhausted

Description: `Receipt::redeemFor` skips unauthorized and zero-balance holders, but forwards every other entry to `_redeem`. Before `exerciseDeadline`, collateral cannot be redeemed. Once earlier holders exhaust `consBacked`, the next non-zero holder therefore causes `ExerciseWindowOpen`, rolling back every earlier redemption in the transaction. This matches the documented atomic batch semantics: no holder loses a claim and the keeper can retry with a smaller batch, but the helper cannot provide best-effort progress and the failed attempt wastes gas.

Recommended Mitigation: If partial-progress batches are preferred, skip a holder when neither settlement leg is currently available. Keep the European pre-expiry revert and allow other `_redeem` failures to propagate:

```
function redeemFor(address[] calldata holders) external nonReentrant {
    for (uint256 i = 0; i < holders.length; i++) {
        address h = holders[i];
        if (notAuthorized(h, msg.sender, Perm.REDEEM)) continue;

        uint256 balance = balanceOf(h);
        if (balance == 0) continue;

        if (isEuro() && block.timestamp < expirationDate()) {
            revert BeforeExerciseWindow();
        }
        if (consBacked == 0 && block.timestamp <= exerciseDeadline()) continue;

        _redeem(h, balance);
    }
}
```

Update the `redeemFor` NatSpec to state that entries with no currently available settlement leg are skipped.

Greeki: Accepted as intentional atomic batch behavior; no code change planned. `IReceipt.redeemFor` already states that only unauthorized and zero-balance holders are skipped and that any redemption revert rolls back the whole batch. `test/Sweep.t.sol::test_RedeemFor_RevertAbortsWholeBatch` pins the reported case.

7.3.7 IOption documents the wrong pair-burn event

Description: IOption::transfer, burn state that pair burns emit Redeemed from the Receipt and nothing from the Option. The implementation instead emits PairBurned from the Option, while Receipt::burn emits nothing.

Recommended Mitigation: Update the IOption::transfer, burn NatSpec to state that the Option emits PairBurned and the Receipt emits nothing. No implementation change is required.

GreekFi: Fixed in [PR46](#)

Cyfrin: Verified. The Option NatSpec now identifies PairBurned as the protocol event emitted by pair burns.

7.3.8 Receipt::sweep can clear accrued fee accounting without reporting the cleared amount

Description: Receipt::sweep resets feeAccrued[token] to its 1-wei floor before transferring the token balance. The Swept event reports only the total transfer, so off-chain accounting cannot determine how much accrued fee was cleared. On-chain accounting remains correct.

Recommended Mitigation: Emit the cleared fee amount, either in Swept or in a dedicated event.

GreekFi: Fixed in [PR43](#)

Cyfrin: Verified. Receipt now emits FeeCleared with the accrued-fee amount removed from accounting during a sweep.

7.3.9 FlashExerciseKeeper is illustrative but is not explicitly marked unsafe for production use

Description: The FlashExerciseKeeper snippet is labeled illustrative, and the surrounding NatSpec warns users to grant Perm.EXERCISE only to audited contracts. However, the example omits production checks and does not explicitly say it must not be deployed as-is. This is a documentation-hardening issue; the protocol contracts behave as specified.

Recommended Mitigation: Add a prominent NOT PRODUCTION READY warning stating that the snippet is pseudocode and must not be used in live deployments.

GreekFi: Fixed in [PR44](#)

Cyfrin: Verified. The keeper example is now explicitly marked as pseudocode that is not production-ready and must not be deployed as written.

7.3.10 IReceipt::redeem amount overload NatSpec misstates the revert for amounts above the caller's balance

Description: IReceipt::redeem(uint256) documents that before exerciseDeadline an amount above the caller's balance is not an error, and that after the deadline it reverts InsufficientPool rather than ERC20InsufficientBalance. Neither holds once a market has more than one holder: Receipt::_redeem caps amount_ against the global consBacked and never against the caller's balance, so it reaches _burn and reverts ERC20InsufficientBalance in both cases. The _redeem dev-comment already hedges the opposite way, so the two doc sites disagree.

Impact: The redeem(uint256) NatSpec is not correct on either side of the deadline, and no funds are affected.

Recommended Mitigation: Correct the IReceipt::redeem(uint256) NatSpec to match the implementation, or clamp amount_ to the caller's balance in Receipt::_redeem.

GreekFi: Fixed in [PR45](#)

Cyfrin: Verified. The redeem amount NatSpec now describes the actual capped pre-deadline burn, full post-deadline burn, and possible revert ordering.

7.3.11 Index key fields in Exercise, Expire, and Swept events

Description: Exercise and Expire do not index either account, while Swept indexes only caller. Indexers cannot filter these logs by holder, keeper, token, or recipient without fetching and decoding every event from each clone.

Impact: No contract behavior or funds are affected. Monitoring and accounting are less efficient.

Recommended Mitigation: Index caller and holder in Exercise; index either account field in Expire, since both are always equal; and index token and recipient in Swept.

GreekFi: Fixed in [PR43](#)

Cyfrin: Verified. Exercise, Expire, and Swept now index the relevant caller, holder, recipient, and token fields.

7.3.12 The public Black-Scholes helper does not document its parameter units

Description: `OracleReceipt::price(uint256,uint256,uint256,uint256,bool)` documents moneyness, but not the other parameters. `timeToExpiry` is expressed in seconds, while moneyness, `vol_`, and `rate_` are WAD-scaled. A direct caller can silently obtain the wrong result by assuming all numeric arguments use the same scale.

Recommended Mitigation: Add `NatSpec` for every parameter, explicitly stating that `timeToExpiry` is in seconds and that moneyness, `vol_`, and `rate_` are WAD-scaled.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. The Black-Scholes helper now documents the units and meaning of every parameter.

7.3.13 `OracleReceipt::price` can publish zero at its moneyness precision boundary

Description: `OracleReceipt::price` computes $m = \text{strike} * 1e18 / \text{spot}$ and then returns $(1e18 - \text{option-Price}) * 1e18$. At extreme moneyness, the WAD calculation can floor the Receipt value to zero. In particular, once `timeToExpiry == 0`, a call with $m == 0$ returns full intrinsic option value and the outer function publishes a zero Morpho price; before that boundary, `lnWad(0)` reverts instead.

Morpho treats a zero collateral price as zero borrowing capacity and permits collateral-denominated liquidation to compute zero repayment. However, outside the separately reported post-deadline valuation issue, this requires the modelled Receipt value to be below WAD resolution and is not credible for the intended WETH/sUSDe deployment profile. This is precision and fail-closed hardening, not a Low-severity loss scenario.

Recommended Mitigation: Handle the boundary explicitly: either calculate moneyness with enough precision to preserve a positive Receipt value or revert when the computation would publish zero. The post-deadline case should continue to be addressed by the separate lifecycle fix.

GreekFi: Fixed in [PR38](#)

Cyfrin: Verified. `OracleReceipt` now reverts instead of publishing a zero collateral price.

7.4 Gas Optimization

7.4.1 Receipt::collectFees loads the same fee accumulator twice

Description: Receipt::collectFees reads feeAccrued[token] once for the zero-value guard and again immediately afterward to calculate the collectible amount. There is no intervening write or external call, and optimized IR retains both mapping storage loads.

Recommended Mitigation: Load feeAccrued[token] into a local variable, use the local for the zero-value guard, and derive the collectible amount from the same local.

GreekFi: Fixed in [PR37](#)

Cyfrin: Verified. Receipt now loads the fee accumulator once and reuses the cached value.

7.4.2 Factory::createOptions, createOptions2 copy read-only batch inputs into memory

Description: Factory::createOptions, createOptions2 are external entry points whose dynamic array parameters are read-only, but the arrays are declared as memory. The ABI decoder therefore copies the complete arrays from calldata before the loops use them. The cost grows with every batched market and salt.

Recommended Mitigation: Declare the external batch parameters as calldata and refactor the shared creation logic so each entry can be consumed without copying the entire arrays into memory. Ensure the internal helper boundary also accepts calldata-compatible input; otherwise the copy is merely moved into each loop iteration.

GreekFi: Fixed in [PR33](#)

Cyfrin: Verified. Factory now keeps read-only creation parameters and batch arrays in calldata through the shared creation path.

7.4.3 Receipt::collectFees reads factory.owner() twice

Description: Receipt::collectFees makes two external calls to factory.owner(), once as the transfer recipient and once for the Fee event:

```
function collectFees(address token) external nonReentrant {
    if (feeAccrued[token] == 0) return; // virgin slot (leg never accrued) - guards the c - 1 below
    uint256 c = feeAccrued[token] - 1; // strip the 1-wei floor + the real collectible
    if (c == 0) return; // only the floor remained; nothing to collect
    feeAccrued[token] = 1;
    IERC20(token).safeTransfer(factory.owner(), c);
    emit Fee(token, factory.owner(), c);
}
```

One call and a local suffices, and it also guarantees the event names the address that received the tokens.

Recommended Mitigation:

```
address to = factory.owner();
IERC20(token).safeTransfer(to, c);
emit Fee(token, to, c);
```

GreekFi: Fixed in [PR37](#)

Cyfrin: Verified. Receipt now reads the Factory owner once and reuses that address for both the transfer and event.